Q. Explain the software Engineering code of ethics and professional practice.

Software Engineering Code of Ethics & Professional Practice (Easy Explanation)

The **Software Engineering Code of Ethics and Professional Practice** is a set of **moral rules and guidelines** that tell software engineers **how to behave responsibly and honestly** while doing their work.

It was created to ensure that software engineers:

- Protect **public safety**
- Act **honestly and fairly**
- Build **reliable and secure software**
- Respect **clients, employers, and society**

📌 Why is this Code Important?

Software today controls:

- Banking systems 🏦
- Medical devices 🏥
- Transportation ✈️
- Government services 🏛️

A small mistake or unethical behavior can cause **huge losses or even loss of life**.
So, this code helps engineers make **right decisions**.

🧭 Main Principles of Software Engineering Code of Ethics

1️⃣ Public Interest

Meaning:
A software engineer must always work for the **benefit and safety of the public**.

Example:
If you find a bug in a hospital software that can show wrong patient data, you **must report and fix it immediately**, even if it delays the project.

2️⃣ Client and Employer

Meaning:
Be honest and loyal to your client and employer **as long as public safety is not harmed**.

Example:
If a client asks you to hide security flaws in an app, you should **refuse**, because users' data could be at risk.

3️⃣ Product

Meaning:
Software engineers should ensure the software is:

- Correct
- Secure
- Tested properly
- High quality

Example:
Before releasing a banking app, you must **test it thoroughly** to avoid money loss due to errors.

4️⃣ Judgment

Meaning:
Engineers should make **independent and honest decisions**, not influenced by pressure or money.

Example:
Even if your boss says "release it fast," you should delay release if the software is unsafe.

5️⃣ Management

Meaning:
Managers should promote **ethical practices** and ensure team members follow them.

Example:
A project manager should **not force developers to skip testing** just to meet deadlines.

6️⃣ Profession

Meaning:
Engineers should improve the **reputation of the software profession**.

Example:
Do not use pirated software or copy someone else's code illegally.

7️⃣ Colleagues

Meaning:
Be fair, supportive, and respectful to other software engineers.

Example:
Help a junior developer understand code instead of blaming them for mistakes.

8️⃣ Self

Meaning:
Software engineers should:

- Keep learning new skills
- Improve knowledge
- Work ethically throughout their career

Example:
Learn new programming languages and security practices to stay updated.

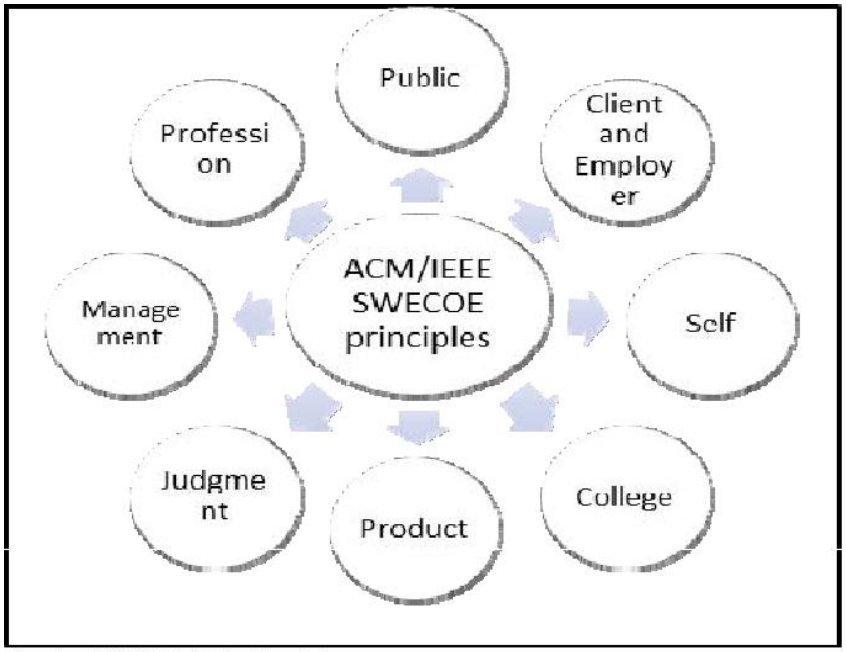


Fig. 1. ACM/IEEE 8 Principles

Q. explain software process activities.

Software Process Activities (Easy Explanation with Examples)

Software process activities are the **main steps followed to develop software** from start to end. They tell us **what work is done** while building software in a systematic way.

Most software processes are divided into **four main activities**.

🔁 1. Software Specification (What to build?)

👉 Meaning

This activity is about **understanding and deciding what the software should do**.

👉 **What happens here?**

- Talk to users and clients
- Understand their needs
- Write requirements clearly

👉 **Example**

Before making a **college management system**, we decide:

- Student login
- Attendance system
- Marks entry
- Fee payment

📌 Output: **Requirement Specification Document (SRS)**

🛠️ **2. Software Design & Implementation (How to build?)**

👉 **Meaning**

This activity is about **designing and coding the software**.

👉 **What happens here?**

- Design system architecture
- Decide database structure
- Write program code

👉 **Example**

For a **shopping app**:

- Design pages (login, cart, payment)
- Create database tables (users, products)
- Write code using Java, Python, etc.

📌 Output: **Working software (code)**

🧪 **3. Software Validation (Is it correct?)**

👉 **Meaning**

This activity checks whether the software **works correctly and meets user requirements**.

👉 **What happens here?**

- Testing the software
- Finding and fixing bugs
- Verifying expected output

👉 **Example**

Testing a **banking app**:

- Check login security
- Verify money transfer
- Test error handling

📌 Output: **Tested and verified software**

🔧 **4. Software Evolution / Maintenance (Change & improve)**

👉 **Meaning**

After delivery, software needs **updates, fixes, and improvements**.

👉 **What happens here?**

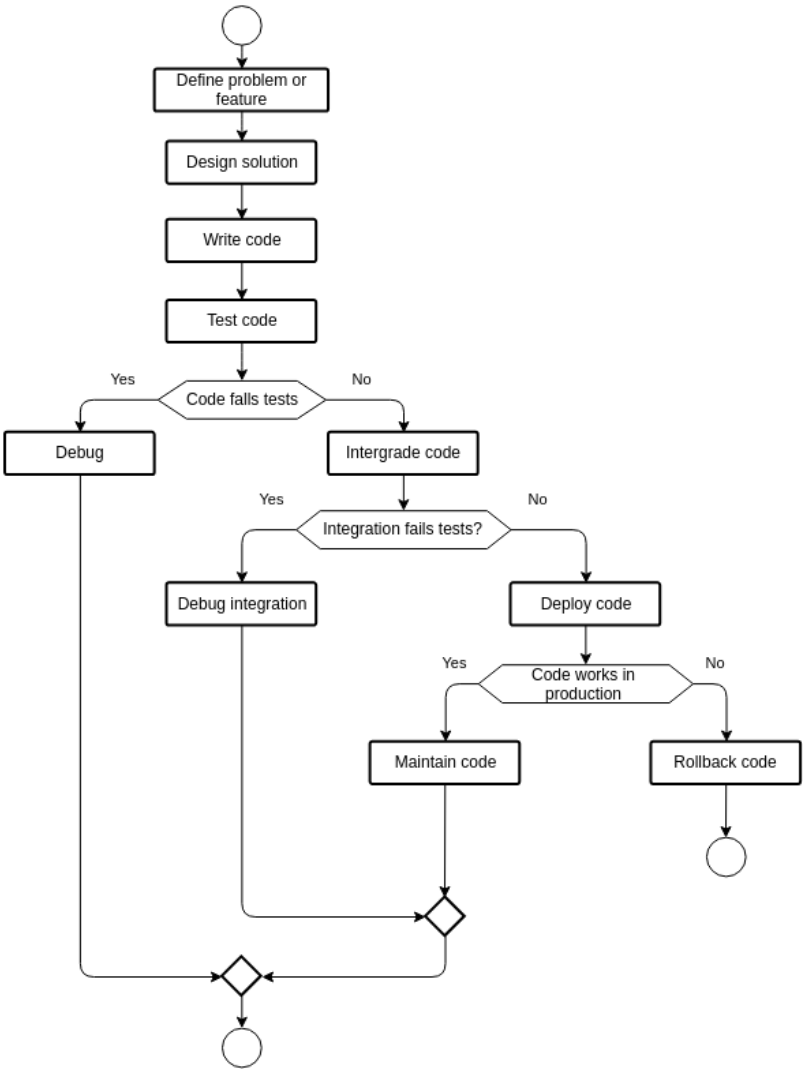
- Fix bugs
- Add new features
- Improve performance

👉 **Example**

Updating a **mobile app**:

- Add dark mode
- Fix crash issues
- Improve speed

📌 Output: **Updated software versions**



Q. Demonstrate your understanding of umbrella activities of a Software process

Umbrella Activities of a Software Process (Easy Explanation)

Umbrella activities are the activities that are performed **throughout the entire software development process**, not just at one single phase.
They **support and control** the main software process activities (like design, coding, testing, etc.).

They are called **"umbrella"** activities because they **cover all phases** of the software process from start to end.

🧠 **Definition (Exam-ready)**

Umbrella activities are supporting activities that are applied continuously across all phases of the software process to ensure quality, control, and proper management of software development.

☂️ **Major Umbrella Activities (with Easy Examples)**

1️⃣ Software Project Management

Meaning:
Planning, scheduling, and controlling the software project.

Includes:

- Time estimation
- Cost estimation
- Resource allocation

Example:
A project manager decides deadlines and assigns tasks to developers.

2️⃣ Risk Management

Meaning:
Identifying and reducing risks that can affect the project.

Examples of risks:

- Delay in delivery
- Technology failure
- Lack of skilled staff

Example:
Keeping backup developers in case a key programmer leaves.

3️⃣ Software Quality Assurance (SQA)

Meaning:
Ensuring software meets quality standards.

Includes:

- Reviews
- Audits
- Quality checks

Example:
Checking whether coding standards are followed.

4️⃣ Formal Technical Reviews

Meaning:
Reviewing software work products to find errors early.

Includes review of:

- Requirements
- Design
- Code

Example:
Senior developers review code before final submission.

5️⃣ Measurement (Metrics)

Meaning:
Collecting data to measure progress and quality.

Examples:

- Number of bugs
- Time taken for tasks
- Code size

Example:
Tracking how many defects are found during testing.

6️⃣ Software Configuration Management (SCM)

Meaning:
Managing changes in software and maintaining versions.

Includes:

- Version control
- Change control
- Backup

Example:
Using Git to manage different versions of code.

7️⃣ Reusability Management

Meaning:
Using existing software components to save time and cost.

Example:
Reusing a login module in multiple projects.

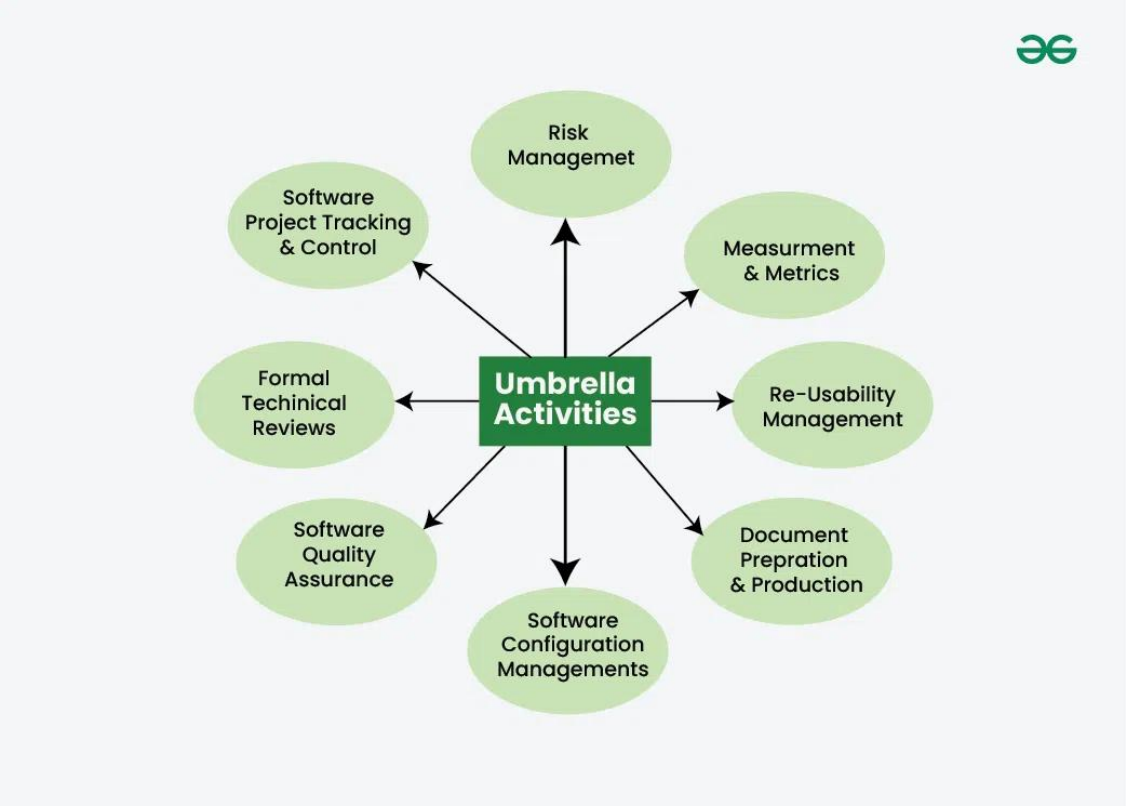
8️⃣ Work Product Preparation & Documentation

Meaning:
Creating and maintaining proper documents.

Includes:

- SRS
- Design documents
- User manuals

Example:
Writing a user guide for a mobile application.



Q. List and describe qualities of good software

Qualities of Good Software (Easy Explanation with Examples)

Good software is software that **works correctly, is easy to use, reliable, and meets user needs**. In Software Engineering, a software product is considered *good* when it satisfies both **users** and **developers**.

🧠 Definition (Exam-Ready)

Qualities of good software are the characteristics that determine how well a software system performs, how easy it is to use, maintain, and how efficiently it meets user requirements.

🌟 Main Qualities of Good Software

1️⃣ Correctness

Meaning:
Software should give **correct output for all valid inputs**.

Example:
A calculator app must always show the right answer for calculations.

2️⃣ Reliability

Meaning:
Software should work **without failure** for a long time.

Example:
ATM software should not crash while withdrawing money.

3️⃣ Efficiency

Meaning:
Software should use **minimum resources** (CPU, memory, time).

Example:
A mobile app should load fast and not drain battery.

4️⃣ Usability (User-Friendliness)

Meaning:
Software should be **easy to learn and easy to use**.

Example:
A website with clear buttons and simple navigation.

5️⃣ Maintainability

Meaning:
Software should be **easy to modify and fix**.

Example:
Adding a new feature to an existing app without breaking old features.

6️⃣ Portability

Meaning:
Software should work on **different platforms**.

Example:
An application running on Windows, Linux, and macOS.

7️⃣ Security

Meaning:
Software should protect **data from unauthorized access**.

Example:
Using password protection and encryption in banking apps.

8️⃣ Scalability

Meaning:
Software should handle **increased users or data**.

Example:
An online shopping site handling more users during festival sales.

9️⃣ Reusability

Meaning:
Parts of software can be **used again in other projects**.

Example:
Using the same login module in different applications.

🔟 Testability

Meaning:
Software should be easy to **test and verify**.

Example:
Modular code that allows testing each feature separately.

Q. With neat sketch explain Verification and Validation model used for Software Engineering

- **1. Introduction**

In Software Engineering, **Verification and Validation (V&V)** are two very important activities used to ensure that:

- The software is **built correctly**
- The software **meets user requirements**

To clearly represent these activities, we use the **Verification and Validation Model**, also called the **V-Model**.

🔹 2. Meaning of Verification and Validation

✅ Verification

"Are we building the product right?"

- Focuses on **process**
- Checks documents, design, and code
- Done **before execution of code**
- Uses **reviews, inspections, walkthroughs**

Example:
Checking whether the design follows the requirements given in the SRS document.

✅ Validation

"Are we building the right product?"

- Focuses on **product**
- Checks actual working software
- Done **after code execution**
- Uses **testing methods**

Example:
Testing whether the final software actually solves the user's problem.

🔹 3. What is the V-Model?

The **V-Model** is a software development model where:

- **Verification phases** are on the **left side**
- **Validation (testing) phases** are on the **right side**
- **Coding** is done at the bottom

Each development phase has a **corresponding testing phase**.

Phases of V-Model (Detailed Explanation)

🟦 LEFT SIDE – VERIFICATION PHASES (Development)

1️⃣ Requirement Analysis

- User requirements are collected
- Functional and non-functional requirements are defined
- SRS (Software Requirement Specification) document is prepared

📌 **Verification Activity:**
Review of SRS to check clarity and completeness

📌 **Corresponding Validation Phase:**
➡️ **Acceptance Testing**

Example:
Requirement: “User should be able to transfer money securely.”

2️⃣ System Design

- Overall system architecture is designed
- Hardware and software requirements are decided

📌 **Verification Activity:**
System design review

📌 **Corresponding Validation Phase:**
➡️ **System Testing**

Example:
Designing modules like login, transaction, database, security.

3️⃣ High-Level Design (HLD)

- System is divided into modules
- Interfaces between modules are defined

📌 **Verification Activity:**
HLD review

📌 **Corresponding Validation Phase:**
➡️ **Integration Testing**

Example:
Interaction between login module and transaction module.

4️⃣ Low-Level Design (LLD)

- Detailed logic of each module is designed
- Algorithms, data structures are defined

📌 **Verification Activity:**
LLD inspection

📌 **Corresponding Validation Phase:**
➡️ **Unit Testing**

Example:
Detailed logic of money transfer function.

5️⃣ Coding (Bottom of V)

- Actual programming is done
- Code is written using programming languages

📌 **Output:**
Executable software modules

🟩 RIGHT SIDE – VALIDATION PHASES (Testing)

6️⃣ Unit Testing

- Individual modules are tested separately
- Done by developers

Example:
Testing login module alone.

7️⃣ Integration Testing

- Combined modules are tested together
- Checks data flow between modules

Example:
Testing login + transaction modules together.

8️⃣ System Testing

- Entire system is tested as a whole

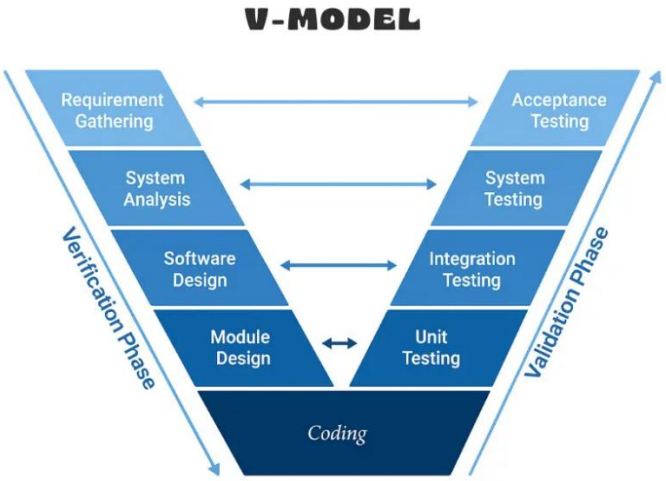
- Functional and non-functional testing is done

Example:
Testing performance, security, and reliability of banking system.

👣 Acceptance Testing

- Performed by client or end user
- Confirms software meets user requirements

Example:
Bank manager tests system before final approval.



Q. Why is Requirements Engineering (RE) considered a critical stage of the Software Process? Discuss the three main activities involved in the requirements engineering process.

Why is Requirements Engineering (RE) considered a critical stage of the Software Process?

📌 Meaning (in simple words)

Requirements Engineering is the process of **finding out, analyzing, documenting, and managing what the user really wants** from the software.

It is considered **critical** because:

👉 If requirements are **wrong or unclear**, the **entire software will be wrong**, even if coding and testing are perfect.

🔥 Why Requirements Engineering is a Critical Stage

1️⃣ Foundation of the Entire Software

- All later stages (design, coding, testing) depend on requirements.
- Any mistake here gets **multiplied** in later stages.

📌 *Example:*
If a banking app forgets to specify "OTP-based security", fixing it later becomes costly.

2️⃣ Cost of Errors is Very High Later

- Errors found during requirements stage are **cheap to fix**.
- Errors found after delivery are **very expensive**.

📌 *Fact often quoted in exams:*
Fixing an error after delivery may cost **10–100 times more** than fixing it during requirements stage.

3️⃣ Avoids Misunderstanding Between User and Developer

- Users may not explain needs clearly.
- Developers may assume wrong things.

Requirements Engineering ensures **clear communication**.

📌 *Example:*
User says "fast system" → RE converts it into measurable terms like "response time < 2 seconds".

4️⃣ Helps in Proper Planning

- Correct requirements help in:
  - Cost estimation
  - Time estimation
  - Resource planning

5️⃣ Improves Software Quality & User Satisfaction

- Clear requirements = right product
- Leads to **less rework**, **better quality**, and **happy users**

🔄 Three Main Activities of the Requirements Engineering Process

The Requirements Engineering process mainly includes **three core activities**:

1️⃣ Requirements Elicitation & Analysis

(Understanding what the user wants)

🔹 Meaning

This activity focuses on **gathering requirements** from stakeholders and analyzing them.

🔹 Techniques Used

- Interviews
- Questionnaires
- Meetings
- Observation
- Brainstorming

🔹 What is done?

- Identify user needs
- Resolve conflicts between requirements
- Check feasibility (technical, economic)

📍 *Example:*
For an **online exam system**, requirements include:

- Student login
- Time limit for exam
- Auto result generation

2️⃣ Requirements Specification

(Writing requirements clearly)

🔹 Meaning

All gathered requirements are **documented clearly and formally**.

🔹 Output

- **SRS (Software Requirement Specification) document**

🔹 SRS includes:

- Functional requirements
- Non-functional requirements (performance, security, reliability)
- Constraints and assumptions

📍 *Example:*
"The system shall allow a maximum of 10,000 users to login simultaneously."

3️⃣ Requirements Validation

(Checking correctness of requirements)

🔹 Meaning

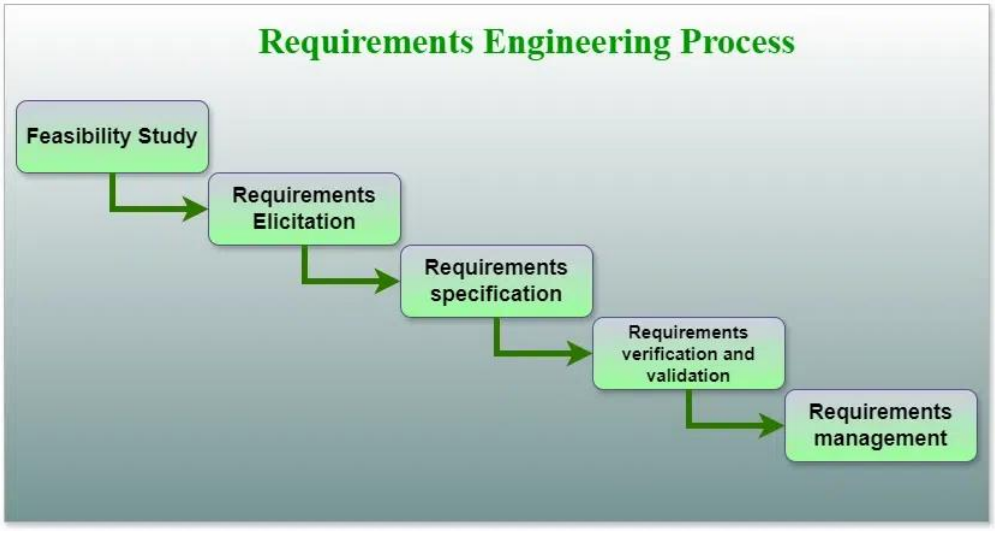
This activity ensures that:

- Requirements are **correct**
- Requirements are **complete**
- Requirements reflect **actual user needs**

🔹 Validation Methods

- Reviews
- Walkthroughs
- Prototyping
- Test-case generation

📍 *Example:*
Showing the SRS or prototype to the client and confirming:
"Is this exactly what you want?"



Q. what are the four important attributes that all professional software should have? state and explain.

Four Important Attributes of Professional Software

(Easy explanation + examples + diagram — exam-oriented)

Professional software is not judged only by whether it works. It must also meet **quality attributes** that make it usable, reliable, and valuable in the real world.

According to Software Engineering principles, **all professional software should have these four key attributes**:

1️⃣ Maintainability

🔹 Meaning

The software should be **easy to modify, fix, and improve** when requirements change.

🔹 Why it is important

- Requirements change over time
- Bugs must be fixed
- New features must be added

If software is hard to maintain, it becomes **costly and risky**.

🔹 Example

A college management system where:

- New subjects can be added easily
- Fee rules can be changed without rewriting the whole program

---

2️⃣ Dependability & Security

🔹 Meaning

The software should be:

- **Reliable** (works correctly without failure)
- **Safe** (does not cause damage)
- **Secure** (protects data from unauthorized access)

🔹 Why it is important

Many systems handle **money, health, or personal data**.

🔹 Example

- ATM software must never lose transaction data
- A banking app must protect user passwords and balance details

---

3️⃣ Efficiency

🔹 **Meaning**

The software should make **efficient use of system resources**, such as:

- CPU time
- Memory
- Network bandwidth

🔹 **Why it is important**

Inefficient software:

- Runs slowly
- Wastes resources
- Gives poor user experience

🔹 **Example**

- A mobile app that loads quickly
- A website that handles thousands of users without slowing down

---

4️⃣ **Acceptability**

🔹 **Meaning**

The software should be **acceptable to users**, which means:

- Easy to use
- Understandable
- Compatible with user environment

🔹 **Why it is important**

Even correct software will fail if users **do not like or understand it**.

🔹 **Example**

- Simple user interface
- Clear instructions
- Works on the user's operating system and devices

Explain different task regions of spiral model with diagram.

Spiral Model – Task Regions Explained (with Diagram)

The **Spiral Model** is a **risk-driven software process model** that combines:

- **Iterative development** (like Incremental model)

- **Systematic control** (like Waterfall model)

In the spiral model, software is developed in **loops (spirals)**.
👉 **Each loop represents one phase of development**, and **each loop is divided into four important task regions**.

🔄 What are Task Regions?

Task regions are the **types of activities performed in each spiral loop**.
They help in **planning, risk handling, development, and customer feedback**.

🧩 The Four Task Regions of the Spiral Model

1️⃣ Planning

📌 *What should be done in this iteration?*

🔹 Meaning

This region defines:

- Objectives of the current iteration
- Alternatives and constraints
- Resources, cost, and schedule

🔹 Activities include:

- Requirement identification
- Project planning
- Effort and cost estimation

🔹 Example

For an **online shopping system**:

- Decide to develop **login and product display** in this iteration.

2️⃣ Risk Analysis

📌 *What can go wrong? How to reduce risk?*

🔹 Meaning

This is the **most important region** of the spiral model.
All possible **technical, cost, schedule, and performance risks** are identified and reduced.

🔹 Activities include:

- Risk identification
- Risk evaluation
- Prototyping to reduce risks

🔹 **Example**

- Risk: Payment gateway may fail
- Solution: Build a **prototype** or test API before full implementation

---

3️⃣ **Engineering**

📌 *Build and verify the product*

🔹 **Meaning**

Actual **development and testing** of software is done here.

🔹 **Activities include:**

- Design
- Coding
- Unit testing
- Integration testing

🔹 **Example**

- Coding login module
- Testing login validation and database connection

4️⃣ **Evaluation (Customer Evaluation)**

📌 *Is the customer satisfied?*

🔹 **Meaning**

The developed product or prototype is **reviewed by the customer or stakeholders**.

🔹 **Activities include:**

- Customer feedback
- Review of deliverables
- Decision to proceed to next loop

🔹 **Example**

- Customer checks login module and suggests UI improvements

Q. Describe incremental approach for software development.

What is the Incremental Approach?

The **Incremental Approach** is a software development method in which the system is **developed and delivered in small parts (increments)** instead of building the entire system at once.

Each increment:

- Adds **new features**
- Is **fully tested**
- Is **usable by the customer**

🧠 **Definition (Exam-Ready)**

The incremental approach is a software development model where the complete system is developed through a series of small, functional increments, with each increment adding new capabilities to the software.

🔄 **How the Incremental Approach Works**

1. The **core (basic) functionality** is developed first

2. Additional features are added in **successive increments**
3. Each increment goes through:
    - Requirement analysis
    - Design
    - Coding
    - Testing
4. The customer can **use the software early** and give feedback

---

🧩 **Phases in Each Increment**

Every increment follows a **mini-SDLC**:

- Requirements
- Design
- Implementation
- Testing

Example: Online Shopping System

Increment	Features Added
Increment 1	User registration & login
Increment 2	Product browsing
Increment 3	Cart & order placement
Increment 4	Payment & delivery tracking

Each increment is delivered and tested before moving to the next.

✅ Advantages of Incremental Approach

✓ Early delivery of working software
✓ Easy to accommodate changes
✓ Reduced project risk
✓ Continuous user feedback
✓ Easier testing and debugging

❌ Disadvantages of Incremental Approach

❌ Requires good planning and design
❌ Total cost may increase
❌ Not suitable if system is very small or tightly coupled